

Milestone 3: Backend API Integration & Frontend UI Development

1. Backend API Integration (Theory)

The ExoHabitAI backend is designed as a RESTful service using the **Flask** framework. This choice was dictated by the need for a lightweight, extensible server capable of performing low-latency machine learning inference in a Python-native environment.

Core Components:

- **Flask Framework:** Handles the HTTP request/response cycle, routing, and JSON serialization.
- **Joblib Serialization:** Used to load pre-trained scikit-learn and XGBoost pipelines efficiently.
- **Data Validation (`utils.py`):** Implements a robust validation layer to ensure that incoming JSON payloads contain all the required astrophysical features (e.g., `orbital_period`, `star_temp_k`) before passing them to the model.
- **Endpoint Architecture:**
 - `/predict` (POST): Accepts exoplanet parameters and returns a classification (e.g., "Rocky-Habitable") along with a confidence score.
 - `/rank` (POST): Accepts a batch of exoplanet data and sorts them based on their habitability probability score, allowing for comparison between multiple terrestrial candidates.

Inference Workflow:

1. The API receives a POST request with a JSON body.
 2. The `validate_prediction_input` utility checks for numerical integrity and feature presence.
 3. The input is transformed into a `pandas` DataFrame, mapped with the identical feature engineering steps used during training (e.g., star class one-hot encoding).
 4. The ensemble model performs a `predict_proba` call to calculate the likelihood for each habitability category.
 5. The result is returned as a structured JSON object.
-

2. Frontend UI Development (Theory)

The frontend is a single-page application (SPA) focused on interactive data exploration. It provides a visual interface for researchers to "design" an exoplanet and receive immediate feedback from the ML models.

Key Technologies:

- **React.js:** Chosen for its efficient state management (using `useState` and `useEffect`), ensuring that UI components respond instantly to input changes.
- **Vanilla CSS:** Applied for a custom, professional "dark space" aesthetic, utilizing modern Flexbox and Grid layouts for responsiveness.
- **Axios:** Used for asynchronous communication with the backend API.
- **Chart.js:** Provides visual breakdowns of prediction probabilities, allowing users to see not just the top result, but the competing likelihoods of other planetary classifications.

User Interaction Model:

1. **Input Grouping:** Parameters are grouped logically into Planetary and Stellar metrics.
 2. **Quick Templates:** Users can click "Earth-like" or "Gas Giant" to pre-fill the form with baseline data, reducing manual entry.
 3. **Real-time Feedback:** Upon submission, the "Predict Habitability" button triggers an API call. The UI shows a loading state and then animates the results onto the screen using cards and progress bars for the confidence scores.
-

Conclusion

The integration of a Flask REST API with a React-based frontend allows for a seamless "Model as a Service" experience. The system is robust, handles errors gracefully, and provides a premium user experience for astrophysical data analysis.